

关节电机培训

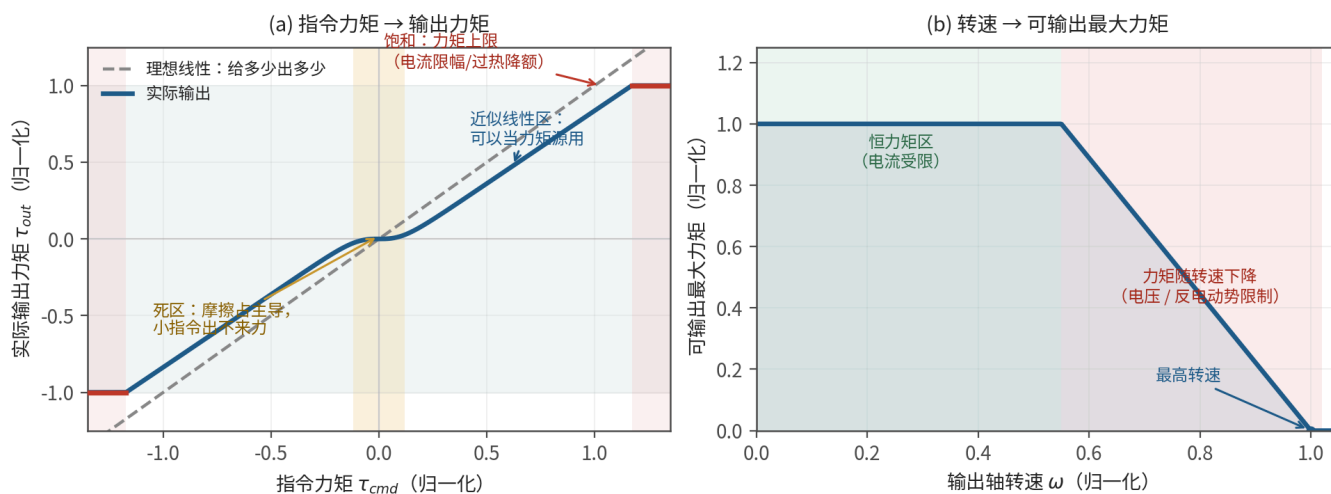
一、关节电机的控制方式与仿真模拟

1.1 关节电机 $\approx$ 力矩源

关节电机的驱动板里已经封装好了 FOC 电流环，上层只需要发指令，电机负责把力矩输出到关节。对上层程序来说，可以近似认为：

你让它输出多大力矩，它就输出多大力矩。

但是必然存在非线性



小力矩时摩擦占主导（死区），大力矩时受电流限制而饱和，高速时又受电压和反电动势限制。所以“给多少出多少”只在中间的近似线性区成立，仿真里也主要按这一段建模，超出范围的部分再用限幅、摩擦和转速-力矩曲线去补。

所以在仿真里：先按理想力矩源跑通，再按需要补限幅、延迟、噪声。

1.2 控制律：MIT 公式

宇树关节电机固件里跑的就是下面这条公式（手册第 4 节）：

4. 电机的混合控制

关节电机作为一个高度集成的动力单元，其内部已经封装了电机底层的控制算法。作为用户，只需要给关节电机发送相关的命令，电机就能完成从接收命令到关节力矩输出的全部工作。

对于电机的底层控制算法，唯一需要的控制目标就是输出力矩。可是对于机器人，我们通常需要给关节设定位置、速度和力矩。这时就需要对关节电机进行混合控制。

宇树科技的关节电机包含如下 5 个控制指令：

- 1. 前馈力矩： τ_{ff} ;
- 2. 期望角度位置： p_{des} ;
- 3. 期望角速度： ω_{des} ;
- 4. 位置刚度： k_p ;
- 5. 速度刚度（阻尼）： k_d 。

在关节电机的混合控制中,使用PD控制器将电机在输出位置的偏差反馈到力矩输出上：

$$\tau = \tau_{ff} + k_p \times (p_{des} - p) + k_d \times (\omega_{des} - \omega)$$

式中， τ 为关节电机的电机转子输出力矩， p 为电机转子的当前角度位置， ω 电机转子的角速度。在实际使用关节电机时，需要注意将电机输出端的控制目标量与发送的电机转子的指令进行换算。

$$\tau = \tau_{ff} + K_p (p_{des} - p) + K_d (\omega_{des} - \omega)$$

参数	SDK 字段	含义
τ_{ff}	cmd.T	前馈力矩：不管位置速度直接施加的力矩，用来抵消重力或让上层直接"用力推"
p_{des}	cmd.Pos	期望位置
ω_{des}	cmd.W	期望速度
K_p	cmd.K_P	位置刚度：相当于虚拟弹簧，越大越"硬"
K_d	cmd.K_W	阻尼：相当于虚拟阻尼器，越大越"黏"、越不容易振荡

于是关节就像"弹簧 + 阻尼器 + 恒力源"。Kp、Kd 不只决定关节去哪，还决定这个关节多软——四足机器人的腿能像弹簧一样缓冲落地，靠的就是这一项，这也是它叫**阻抗控制**的原因。

注意：命令是转子侧的

手册明确说明，这五个参数都是减速器之前、**电机转子侧**的量；而仿真模型和训练出来的策略针对的是关节，也就是输出端。两边的映射关系放到第二节讲，这里先记住"公式是转子侧的"就够了。

1.3 各种模式，都是同一条公式的特例

模式	T	W	Pos	K_P	K_W	效果
力矩模式	≠0	0	0	0	0	持续输出固定力矩，空载会一直加速
零力矩模式	0	0	0	0	0	主动抵消自身摩擦，可以自由拖动

模式	T	W	Pos	K_P	K_W	效果
阻尼模式	0	0	0	0	>0	像粘稠液体，甩一下就停
速度模式	0	ω_{des}	0	0	>0	稳定在某个转速
位置模式	0	0	p_{des}	>0	0	像弹簧拉向目标点
力位混合（MIT）	τ	ω_{des}	p_{des}	>0	>0	四足行走实际使用的模式

实机上还有两点：`mode = 0` 是刹车锁定，`mode = 1` 才是 FOC 闭环；位置模式的目标位置要先做插值、缓慢变化，否则相当于给电机一个大阶跃。

1.4 MuJoCo 里怎么模拟

MuJoCo 的 `<motor>` 执行器就是理想力矩源，`data.ctrl` 就是力矩。所以我们只要把上面那条公式从电机固件里搬到仿真循环里自己算：

```
<motor name="FL_hip" joint="FL_hip_joint" ctrlrange="-33.5 33.5"/>
```

```
# q_adr / dq_adr / act_id 可以提前用 model.jnt_qposadr、
# model.jnt_dofadr、mujoco.mj_name2id 查好；下面都是关节输出侧的量
q = data.qpos[q_adr]
dq = data.qvel[dq_adr]

tau = tau_ff + kp * (q_des - q) + kd * (dq_des - dq)
data.ctrl[act_id] = np.clip(tau, -tau_max, tau_max)

mujoco.mj_step(model, data)
```

要点：

1. `q`、`dq` 相当于实机的反馈，`data.ctrl` 相当于发给电机的力矩指令；
2. `kp`、`kd` 必须用和实机部署完全相同的一组值（输出侧）。队内 `rl_sar` 的 black 配置是 `kp=80`、`kd=3`、力矩上限 `33.5 N·m`，可以拿来对照；
3. 力矩限幅，`ctrlrange` 或代码里 `np.clip` 都可以；
4. 想更像实机，再补一两个周期的指令延迟和编码器噪声。

不要直接用 MuJoCo 自带的 `<position>` 执行器（`data.ctrl` 填目标位置）：内部虽然也是同一套 PD，但加不了前馈力矩，也没法插延迟，和实机程序对不上。

二、关节电机的零点与减速比问题

零点和减速比这个问题比较绕，需要大家稍微耐心一点去理解。先说减速比：以宇树电机为例，它的减速比也是最难处理的；把宇树电机的减速比处理清楚，其他关节电机处理起来就相对简单了。

2.1 先区分转子和输出端



此处需要**特别注意**的是，给电机发送的命令都是针对减速器之前的电机转子，即图 5 中的转轴 1。而不是经过减速之后的输出轴 2。所以在进行实际控制的过程中，一定要注意考虑电机的减速比。在 GO-8010-6 的电机中，**减速比为 6.33**。

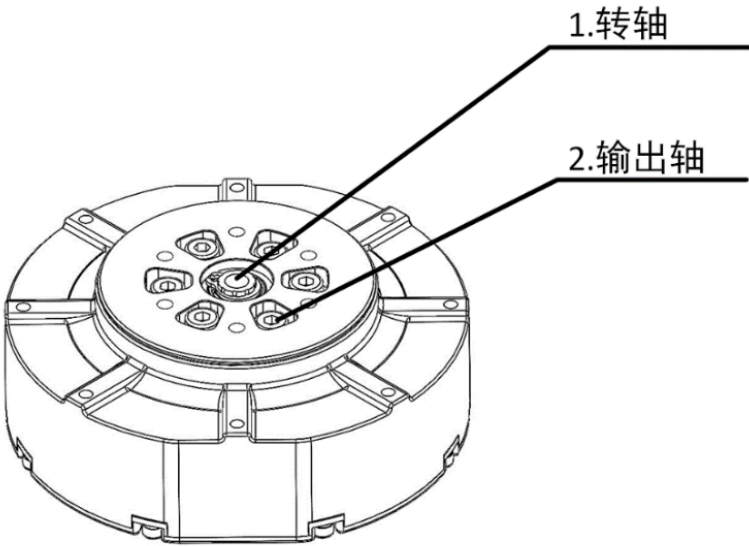


图 4 电机输出端

两者通过减速器连接：**转子转 6.33 圈，输出端才转 1 圈**，所以减速比 $N = 6.33$ 。

2.2 为什么需要映射

1.2 里那条控制律，最终控制的对象是**转子**：`cmd.Pos`、`cmd.W`、`cmd.T`、`cmd.K_P`、`cmd.K_W` 都是转子侧的量，反馈回来的 `data.Pos`、`data.W` 同样是转子侧的量。

而仿真模型和训练出来的策略针对的是**关节**，也就是输出端。所以必须找到一组映射关系，让我们能直接用输出端的量去思考和下发指令。

2.3 映射关系

记输出端（关节）的角度为 q 、角速度为 $\dot{q}$ 、力矩为 τ_{out} ，转子端对应 p 、 ω 、 τ_{rotor} ，减速比 $N = 6.33$ 。

反馈方向（转子 → 输出端）

$$q = \frac{p}{N}, \quad \dot{q} = \frac{\omega}{N}, \quad \tau_{out} = N \tau_{rotor}$$

指令方向（输出端 → 转子）

输出端（关节）	下发的电机命令	关系
目标位置 q_{des}	<code>cmd.Pos</code>	$N q_{des}$
目标速度 $\dot{q}_{des}$	<code>cmd.W</code>	$N \dot{q}_{des}$
前馈力矩 τ_{ff}	<code>cmd.T</code>	τ_{ff}/N
位置刚度 K_p	<code>cmd.K_P</code>	K_p/N^2
阻尼 K_d	<code>cmd.K_W</code>	K_d/N^2

记忆口诀：**位置、速度乘 N ，力矩除 N ，刚度阻尼除 N^2** 。这里 $N^2 \approx 40.07$ 。

代入验证一下：把 $p = Nq$ 、 $\omega = N\dot{q}$ 和上表的命令代回 1.2 的公式，得到

$$\tau_{out} = N \tau_{rotor} = \tau_{ff} + K_p (q_{des} - q) + K_d (\dot{q}_{des} - \dot{q})$$

也就是说，在输出端看到的仍然是同一条 MIT 公式，只是参数都换成了关节侧的值。手册位置模式里写 `cmd.Pos = 3.14 * 6.33`，用的就是这个映射。

想了解原理的同学可以自己推导一遍，不难。

2.4 单圈绝对值编码器与零点

编码器装在**转子上**，宇树电机用的是一颗**单圈绝对值编码器**：

- **绝对值**：上电就能直接读出当前角度，不需要像增量编码器那样先找零；
- **单圈**：它只保证"这一圈内的角度"是绝对的。不掉电的时候它也能累计转了多少圈，但一旦掉电再上电，圈数就丢了，读数只会回到当前这一圈内的 $0 \sim 360^\circ$ 。

编码器有一个固定的零点（读数为 0 的位置）。又因为减速比 $N = 6.33$ ，**转子转 1 圈，输出端只转 $1/6.33$ 圈**，所以：

- 输出端转 1 圈 $\rightarrow$ 转子转 6.33 圈 $\rightarrow$ 编码器零点被经过 6.33 次；
- 输出端相邻两个"编码器零点"之间的距离是 $1/6.33$ 圈。

举个具体的例子。设某一时刻转子正好在编码器零点，我们把这里记作输出端的零点：

1. 输出端转过 $1/6.33$ 圈，转子正好转完一整圈，编码器读数又回到 0；
2. 此时断电再上电，单圈编码器丢掉圈数，读数还是 0；
3. 但输出端已经走过了 $1/6.33$ 圈。

也就是说，同样是"编码器读数为 0"，输出端的位置可以相差 $1/6.33$ 圈。把这个过程重复下去，输出端一圈里就有 6.33 个位置都能被当作零点。这些零点由减速器的装配关系决定，位置是固定的，我们可以把其中任意一个认为是输出端的零点。

为什么是 6.33 而不是整数？提示：减速比实际是 19:3——转子转 19 圈、输出端转 3 圈。所以输出端三圈里正好有 19 个零点，平均一圈 6.33 个，相邻间隔是 $3/19$ 圈（约 56.8° ）。为了方便，后面我们就说"6 个零点"。

2.5 为什么直接用上电读数不行

机器狗每次启动时是趴着的。在这个姿态下开机，每个电机都会给出一个位置值（以下说的都是输出端的位置）：

这个值是**相对于最近经过的那个零点**走过的角度，范围只有 $0 \sim 1/6.33$ 圈（约 $0 \sim 57^\circ$ ）。

因为这些零点在输出端一圈里的位置由装配决定，几乎可以认为是随机的，如果直接把上电时的读数当作零点，12 个电机的零点就是散的、不对称的，四条腿根本对不上。我们希望零点是对称、有规律的。

做法是在软件里加一个偏移量：

- 收到的角度： $q = \text{data.Pos}/N + \text{offset}$
- 下发的角度： $\text{cmd.Pos} = (q_{des} - \text{offset}) \cdot N$

收到加偏移、下发减偏移，这样软件里看到的就是我们想要的关节角。测量出这个 offset、把零点挪到想要的位置，这个过程就叫做**电机的标定**。

请仔细思考上面的内容，再继续往下看.....

2.6 万一"认错零点"

到这里零点确实能挪到想要的位置了，但还有一个问题：**输出端有 6 个零点，启动那一刻，怎么确定用的是标定时的那一个？**

只能靠约定：每次让狗趴在尽量相同的位置启动，这样绝大多数情况下读到的都是同一个零点区间。注意是"几乎"不会认错，不是"一定"。

考虑一个本来就靠近零点边界的电机。标定时它在零点正方向 10° ，某次摆放稍微偏了一点，实际落到了 -10° ，那么上电后编码器读数是 350° 。 10° 变成 350° ，差了 340° ，姿态误差不可能有这么大——唯一的解释是：这次用的基准零点，是标定时那个零点的**相邻零点**。

知道原因以后处理就简单了：相邻两个零点的间距是已知的，输出端 $1/6.33$ 圈（转子侧正好 360° ）。把 offset 加上或者减去这一个间距，就换到了正确的零点上，步骤和标定一样。

这里 10° 、 350° 说的是编码器（转子侧）的读数，换算到输出端要除以 6.33，变成约 1.6° 和 55.3° 。换成输出端来看，读数会跳大约 $1/6.33$ 圈（约 57° ）——这就是判断"认错零点"的特征。

2.7 小结

- 输出端一圈里有 6.33 个固定的候选零点，相邻间隔 $1/6.33$ 圈；
- 上电时电机以"最近经过的那个零点"为基准报角度，所以每次启动会落在哪个零点上并不确定；
- 用软件偏移（收到加 offset、下发减 offset）把零点对齐到想要的位置，这就是标定；
- 用固定的启动姿态尽量避免认错零点；用"角度跳变约一个零点间距"判断是否认错，认错了补一个间距即可。